

# Modeling

Maksym Khavil

2025-07-19

## Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Modelling</b>                            | <b>1</b> |
| 1.1      | Setup . . . . .                             | 1        |
| 1.2      | Baseline Models . . . . .                   | 4        |
| 1.3      | Feature Selection and Engineering . . . . . | 13       |
| 1.4      | Final model . . . . .                       | 28       |
| 1.5      | Assumption Verification . . . . .           | 29       |
| 1.6      | Final Decision Tree . . . . .               | 36       |
| 1.7      | Wrap up . . . . .                           | 37       |

## 1 Modelling

### 1.1 Setup

Let us load the data and libraries.

```
library(stats)
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```
library(rpart)
library(rpart.plot)
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## cov, smooth, var
```

```
library(corrplot)
library(car)
```

```
## Loading required package: carData
```

```
source(here::here("R", "constants.R"))
source(here::here("R", "load_data.R"))
```

```
data <- load_bank_data()
head(data)
```

```
##   age      job marital education default balance housing loan  contact day
## 1  30 unemployed married  primary      no   1787      no   no cellular 19
## 2  33  services married secondary     no   4789     yes  yes cellular 11
## 3  35 management single  tertiary     no   1350     yes   no cellular 16
## 4  30 management married  tertiary     no   1476     yes  yes unknown   3
## 5  59 blue-collar married secondary     no     0     yes   no unknown   5
## 6  35 management single  tertiary     no    747      no   no cellular 23
##   month duration campaign pdays previous poutcome y
## 1   oct       79         1    -1         0 unknown no
## 2   may      220         1   339         4 failure no
## 3   apr      185         1   330         1 failure no
## 4   jun      199         4    -1         0 unknown no
## 5   may      226         1    -1         0 unknown no
## 6   feb      141         2   176         3 failure no
```

We will also define a generic function to run cross-validation. That way we will estimate accuracy on new data, we also collect all predictions to construct confusion matrix and Receiver Operator Curve. We will display AUC ROC, accuracy, f1 score, precision and recall. We model data to better profile, so we choose the metric of performance to be precision, as it is a measure of confidence of model. Nevertheless precision can be abused, so we will also keep an eye on accuracy and f1 score.

```
cv_evaluate_model <- function(model_func, predict_func, weights = NULL, k = 5, seed = 42, arg_data=data)
  set.seed(seed)

  n <- nrow(data)
  folds <- sample(rep(1:k, length.out = n))

  y_true_all <- c()
  y_pred_all <- c()
  y_prob_all <- c()

  for (i in 1:k) {
    train_idx <- which(folds != i)
    test_idx  <- which(folds == i)

    train_data <- arg_data[train_idx, ]
    test_data  <- arg_data[test_idx, ]
    model <- NULL
    if(!is.null(weights)){
      train_weights <- weights[train_idx]
      model <- model_func(y ~ ., data=train_data, weights=train_weights, ...)
    }
    else {
      model <- model_func(y ~ ., data=train_data, ...)
    }

    prob <- predict_func(model, test_data)
    pred <- ifelse(prob > 0.5, "yes", "no")

    y_true_all <- c(y_true_all, test_data[['y']])
    y_pred_all <- c(y_pred_all, pred)
```

```

    y_prob_all <- c(y_prob_all, prob)
  }

  y_true_all <- factor(ifelse(y_true_all == 1, 'no', 'yes'), levels = c("no", "yes"))
  y_pred_all <- factor(y_pred_all, levels = c("no", "yes"))

  cm <- table(Predicted = y_pred_all, Actual = y_true_all)

  tp <- cm["yes", "yes"]
  tn <- cm["no", "no"]
  fp <- cm["yes", "no"]
  fn <- cm["no", "yes"]

  accuracy <- (tp + tn) / sum(cm)
  precision <- tp / (tp + fp)
  recall <- tp / (tp + fn)
  f1 <- 2 * precision * recall / (precision + recall)

  corplot(
    cm,
    is.corr = F, method = "color", tl.col = "black", addCoef.col = "black",
    xlab = "Ground Truth", ylab = "Prediction", mar=c(3,0,3,0),
    title = "Confusion Matrix", cl.pos = 'n', col=c('bisque', 'lightblue')
  )
  mtext("Ground Truth", side = 3, line=1, cex = 1.2)
  mtext("Predictions", side = 2, line=0, cex = 1.2)

  roc_obj <- roc(y_true_all, y_prob_all, levels=c("no", "yes"), direction='<')
  plot(roc_obj, col = "red", main = "ROC Curve")

  legend(
    "bottomright",
    legend = c(
      sprintf("AUC = %.3f", auc(roc_obj)),
      sprintf("F1 = %.3f", f1),
      sprintf("Accuracy = %.3f", accuracy),
      sprintf("Precision = %.3f", precision),
      sprintf("Recall = %.3f", recall)
    )
  )
}

glm_predict_func <- function(model, newdata) predict(model, newdata, type = "response")
rpart_predict_func = function(model, newdata) predict(model, newdata, type = "prob")[, "yes"]

```

Let us define weights as inverse of classes' rates. that way both classes' weights sum up to the same number.

```

weights <- as.double(data$y)
pos_rate <- mean(data$y == 'yes')
weights[data$y == 'yes'] <- 1 / pos_rate
weights[data$y == 'no'] <- 1 / (1 - pos_rate)

```

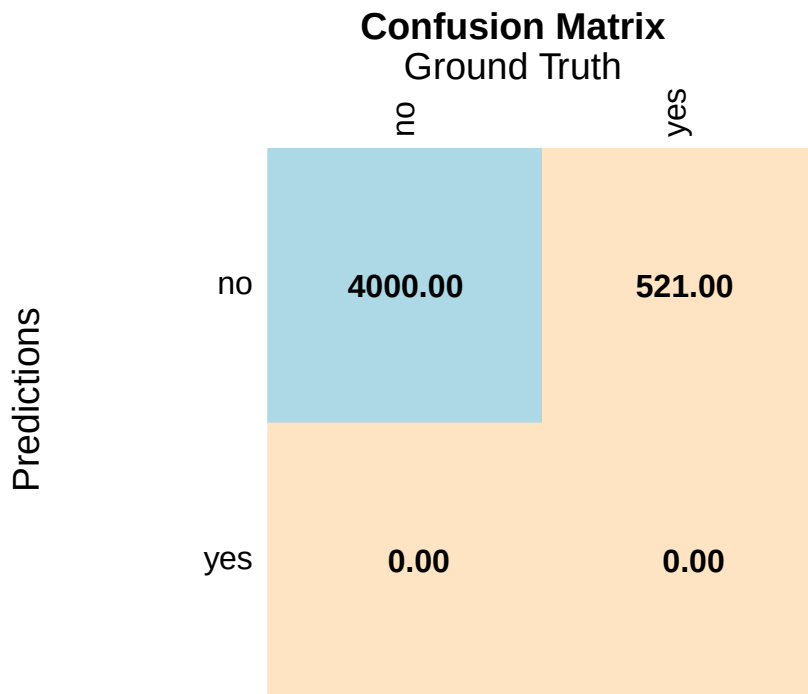
## 1.2 Baseline Models

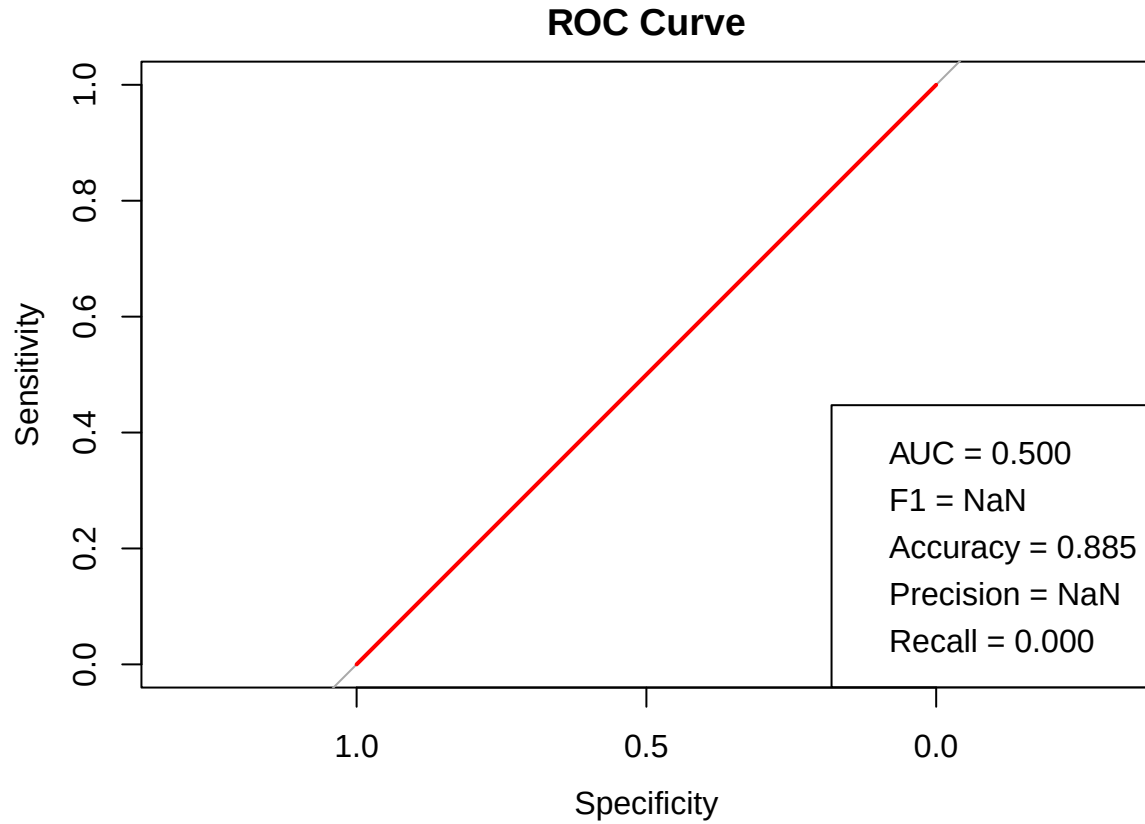
We will create dummy estimator, that predicts the most frequent class, logistic regression and decision tree. Then we will proceed by feature selection and engineering. We hope that we will be able to increase both precision and interpretability of models.

### 1.2.1 Dummy Estimator

Due to class imbalance dummy estimator will have high accuracy.

```
dummy_model <- function(fml, data, ...) {  
  data$y  
}  
  
dummy_predict <- function(train_labels, test_data) {  
  majority_class <- ifelse(names(sort(table(train_labels), decreasing = TRUE))[1] == 'no', 0, 1)  
  rep(majority_class, nrow(test_data))  
}  
cv_evaluate_model(dummy_model, dummy_predict)
```



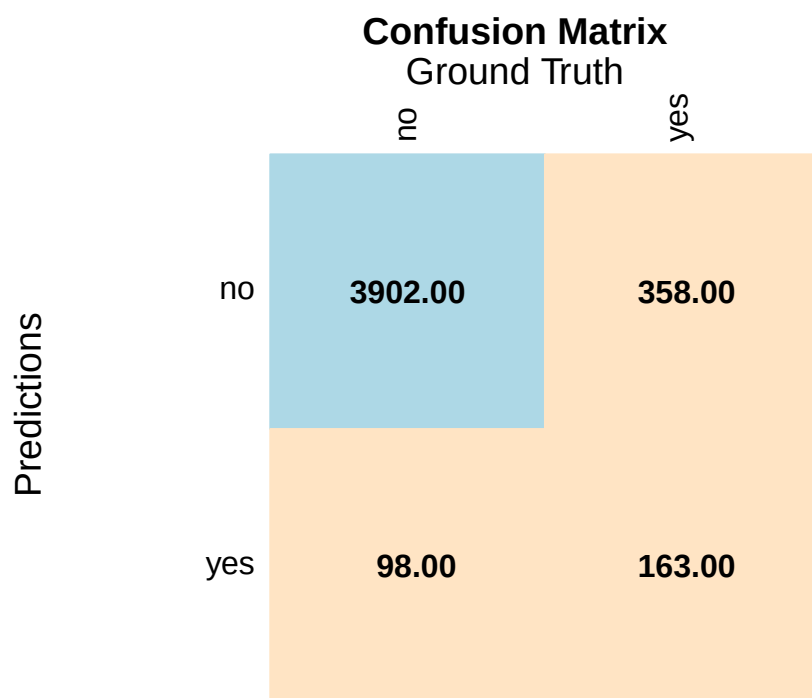


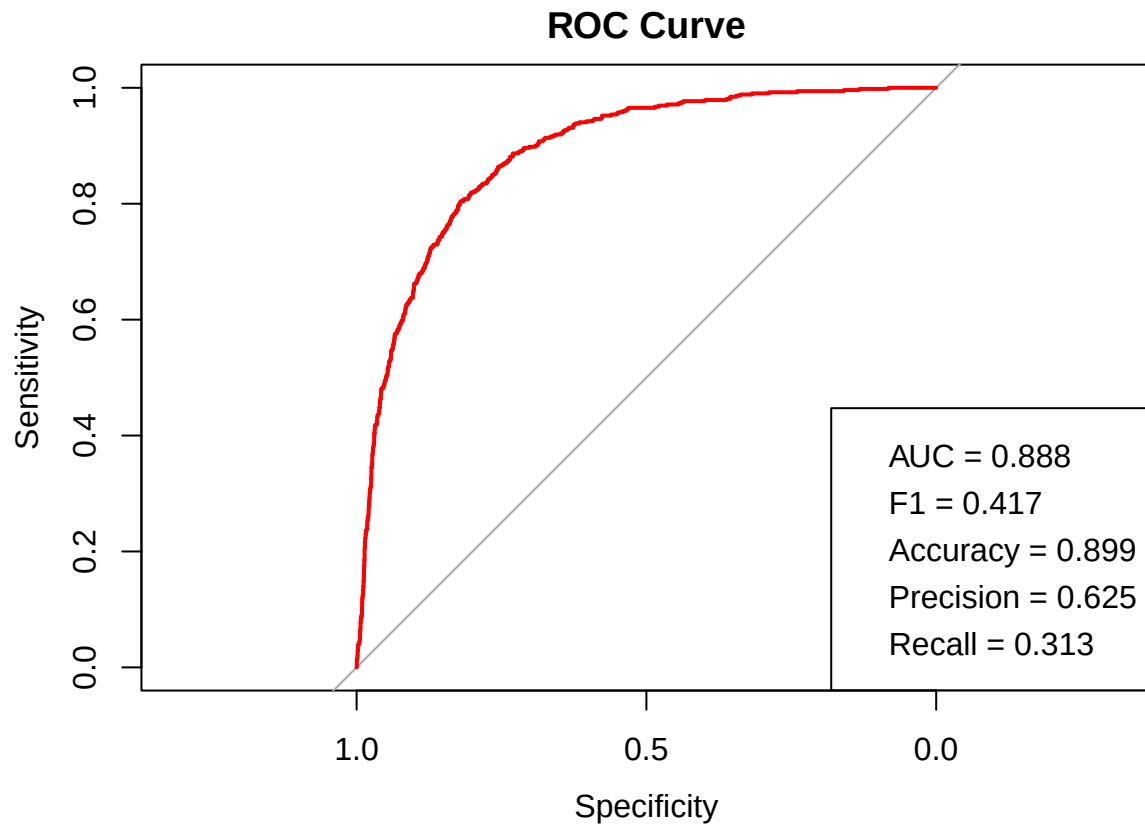
We see that accuracy matches negative rate, and precision and f1 are undefined.

### 1.2.2 Logistic Regression

We will focus on logistic regression, because it gives a clear interpretability during profiling: positive coefficients correspond to better likelihood of person subscribing.

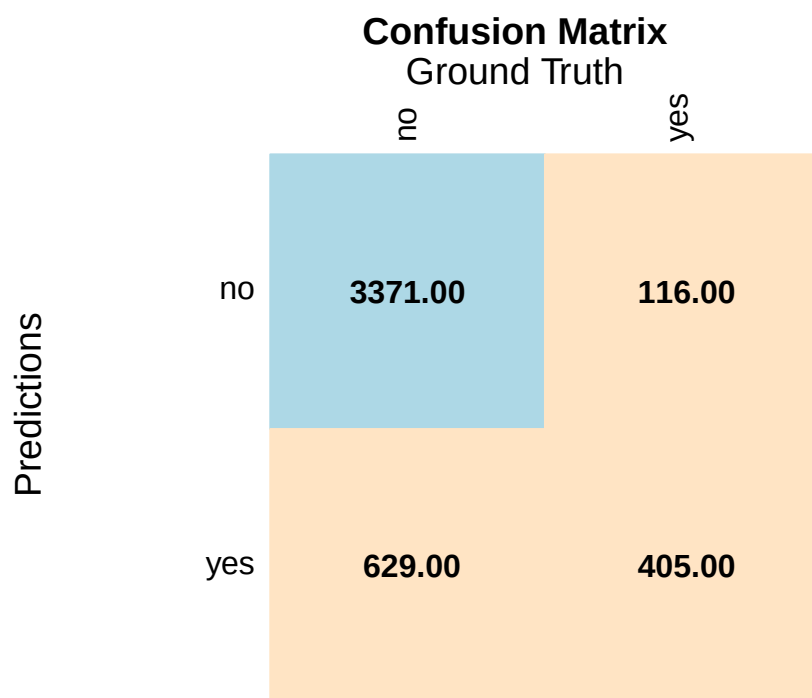
```
cv_evaluate_model(glm, glm_predict_func, family=binomial)
```



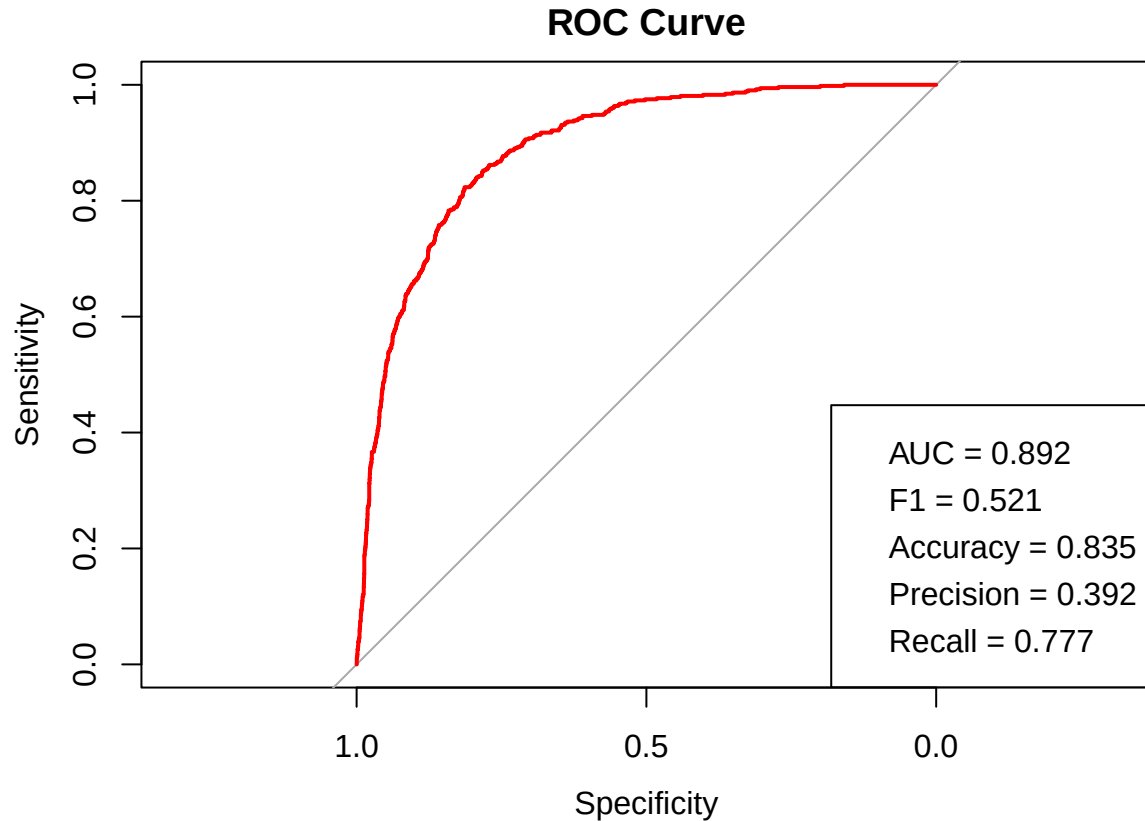


Log. regression strongly prefers negative class, which results in recall of 0.313. Nonetheless accuracy is higher negative rate and precision is greater than 0.625. Let us check weighted model.

```
cv_evaluate_model(glm, glm_predict_func, family=binomial, weights=weights)
```





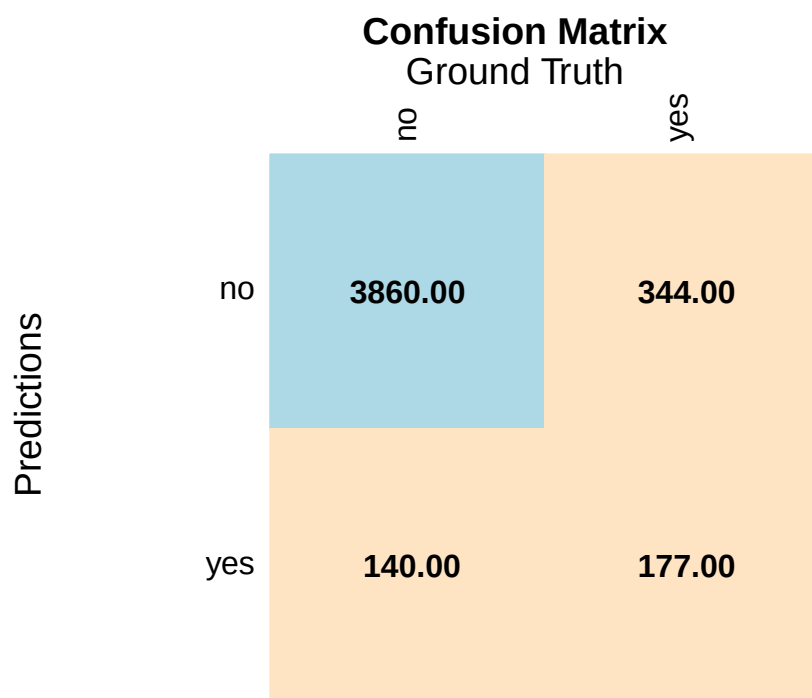


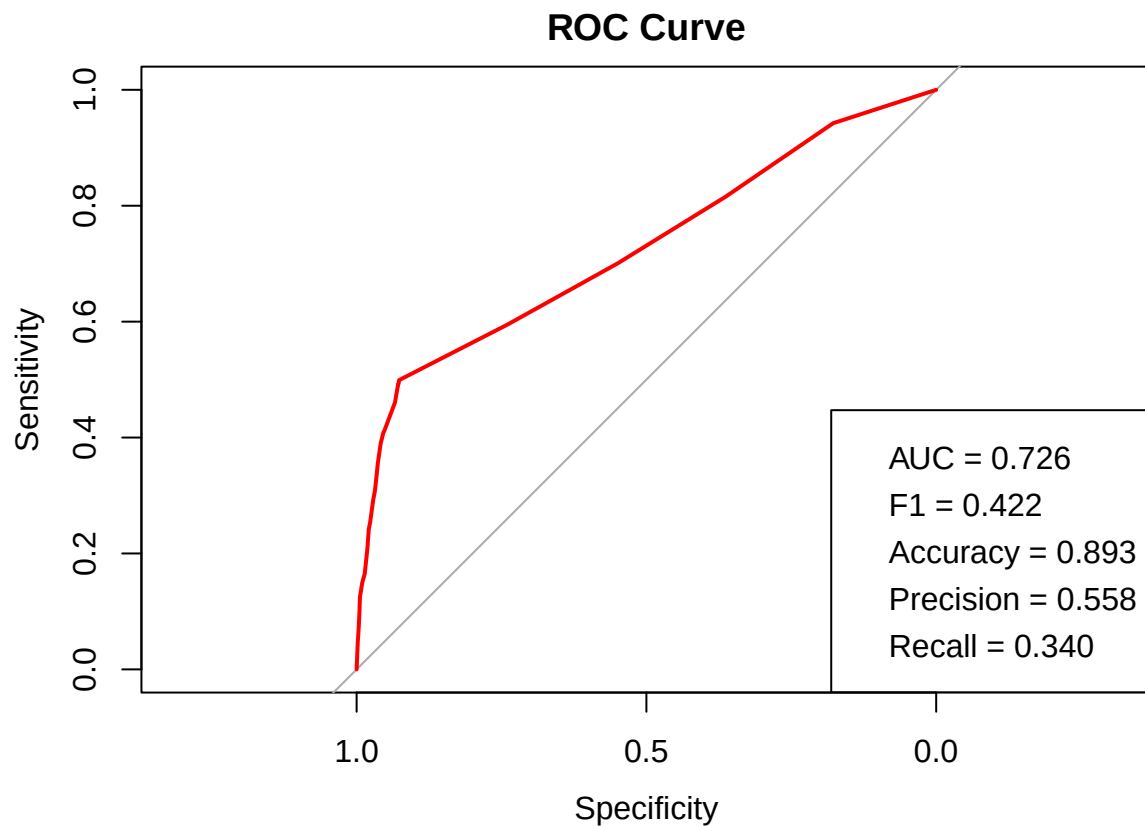
By weighting our samples we have introduced heavy bias towards positive class, so now precision suffers, while recall is up, which results in higher f1 score. This is not the strategy that we try to follow, though it is still important to look at balanced case.

### 1.2.3 Decision Tree Classifier

Our motivation for decision tree classifier is later rule extraction from it. We do not expect it to perform better than linear model, though we hope to see decent result.

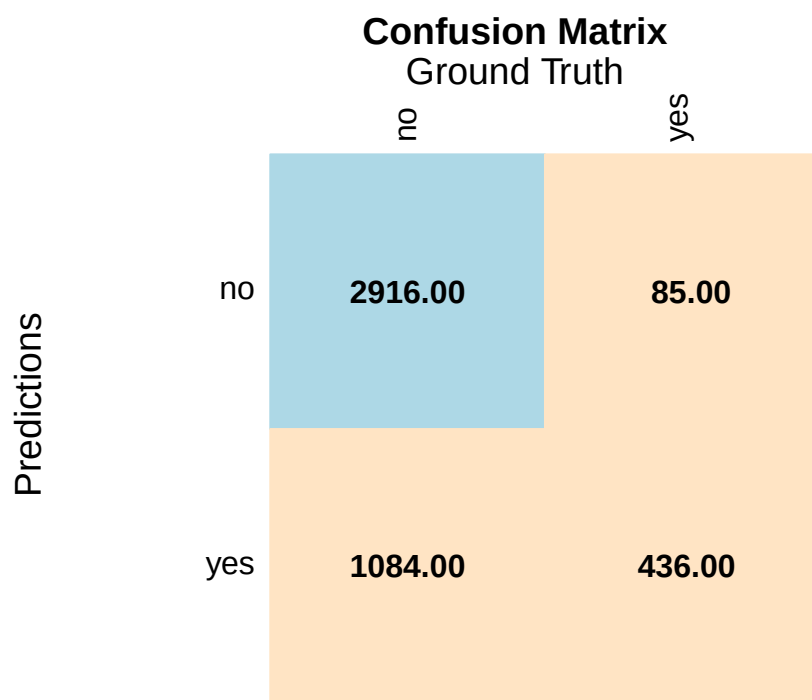
```
cv_evaluate_model(rpart, rpart_predict_func, maxdepth=3, cp=0)
```

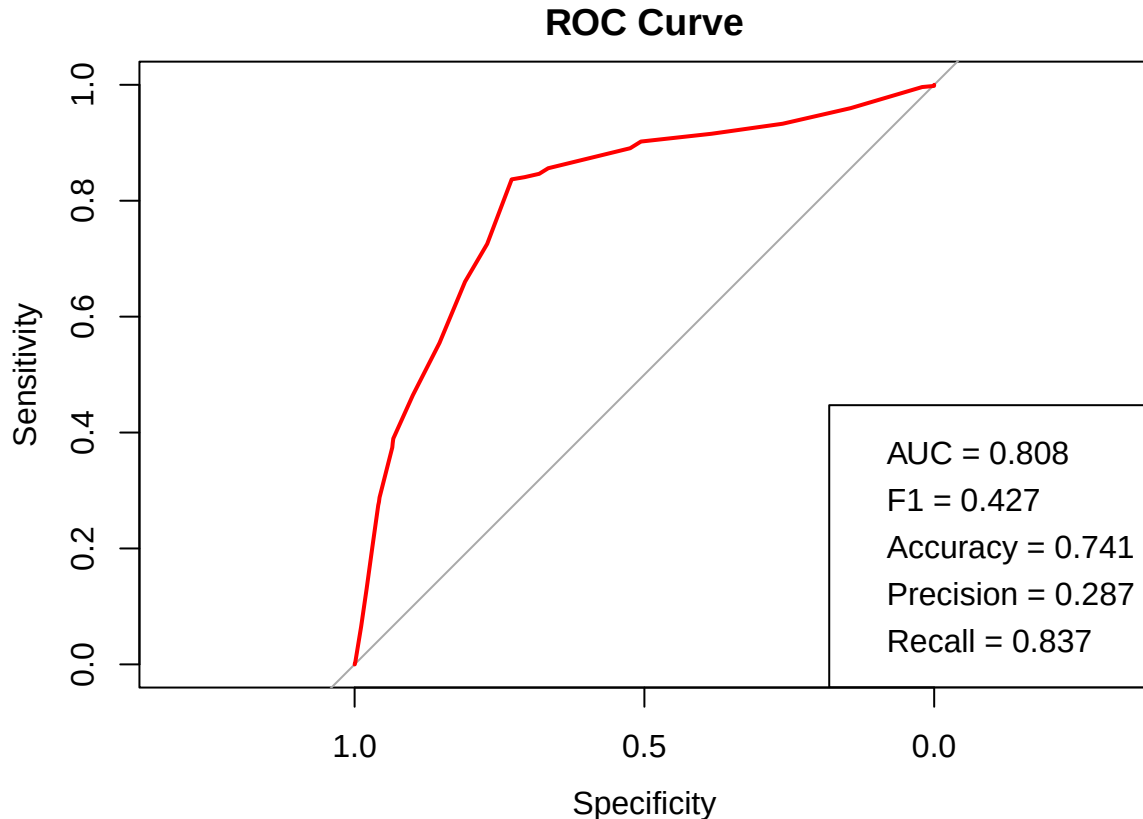




ROC has a sharp angle near the bottom, that indicates, that the trees were in favor of negative class. This is in agreement with low recall values. Precision is lower then the one of unweighted logistic regression, though it still presents some confidence.

```
cv_evaluate_model(rpart, rpart_predict_func, weights = weights, maxdepth=3, cp=0)
```





The sharp angle on ROC moved upward, the model prefers positive class, recall is the highest of all the models we have discussed, while precision is the lowest. The result is very similar to the weighted regularized log regression's one.

### 1.3 Feature Selection and Engineering

During EDA and statistical testing we have confronted several features as being useless (namely **default** and **pdays**). In this section we will use LASSO and greedy step AIC algorithms to find a subset of features that manifests in the best precision. If manual intervention will be needed we will engineer the data ourselves.

#### 1.3.1 LASSO

LASSO is a way to regularize linear models by L1 norm, resulting in sparse solutions. Those features that have zero coefficients can be safely discarded. We will use `glmnet` package LASSO implementation.

```
X <- model.matrix(y ~ ., data = data)[, -1]

y <- data$y
y <- ifelse(y == "yes", 1, 0)

model <- cv.glmnet(X, y, family = "binomial", alpha = 1)
coefs <- coef(model, s = 'lambda.1se')
coefs <- as.matrix(coefs)

# Get variable names with non-zero coefficients
non_zero_vars <- rownames(coefs)[which(coefs != 0)]
non_zero_vars <- setdiff(non_zero_vars, "(Intercept)")
```

```
print(coefs)
```

```
##          lambda.1se
## (Intercept) -2.716503895
## age         0.000000000
## jobblue-collar -0.082114573
## jobentrepreneur 0.000000000
## jobhousemaid 0.000000000
## jobmanagement 0.000000000
## jobretired 0.352548565
## jobself-employed 0.000000000
## jobservices 0.000000000
## jobstudent 0.125039119
## jobtechnician 0.000000000
## jobunemployed 0.000000000
## jobunknown 0.000000000
## maritalmarried -0.099122687
## maritalsingle 0.000000000
## education.L 0.103009826
## education.Q 0.000000000
## education.C 0.000000000
## defaultyes 0.000000000
## balance 0.000000000
## housingyes -0.232198347
## loanyes -0.229418238
## contacttelephone 0.000000000
## contactunknown -0.610381526
## day 0.000000000
## month.L 0.000000000
## month.Q 0.000000000
## month.C 0.000000000
## month^4 -0.918196201
## month^5 0.013622038
## month^6 0.370186286
## month^7 0.000000000
## month^8 0.614578758
## month^9 0.000000000
## month^10 0.000000000
## month^11 0.199691467
## duration 0.003405757
## campaign 0.000000000
## pdays 0.000000000
## previous 0.000000000
## poutcomeother 0.068717871
## pcomesuccess 2.105386702
## poutcomeunknown -0.178414841
```

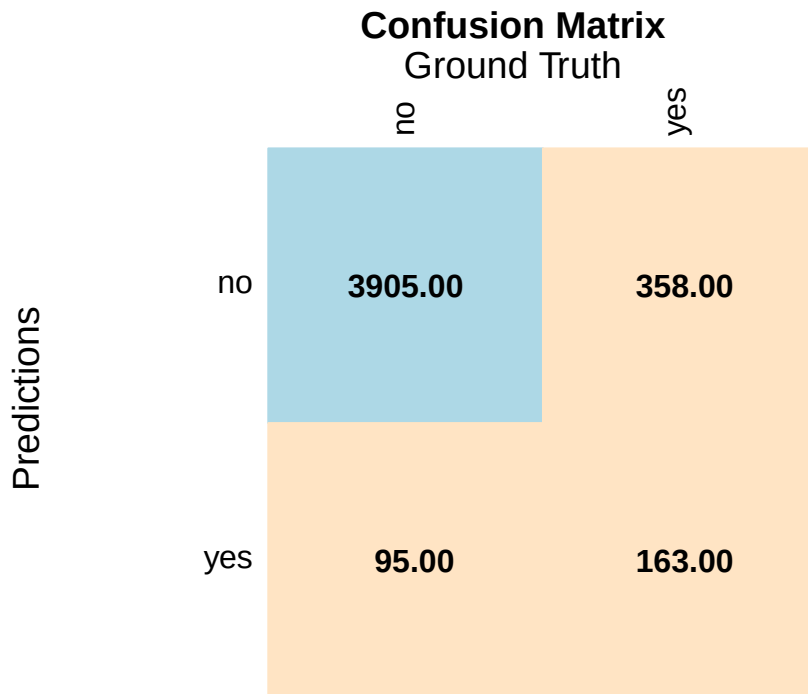
```
non_zero_vars
```

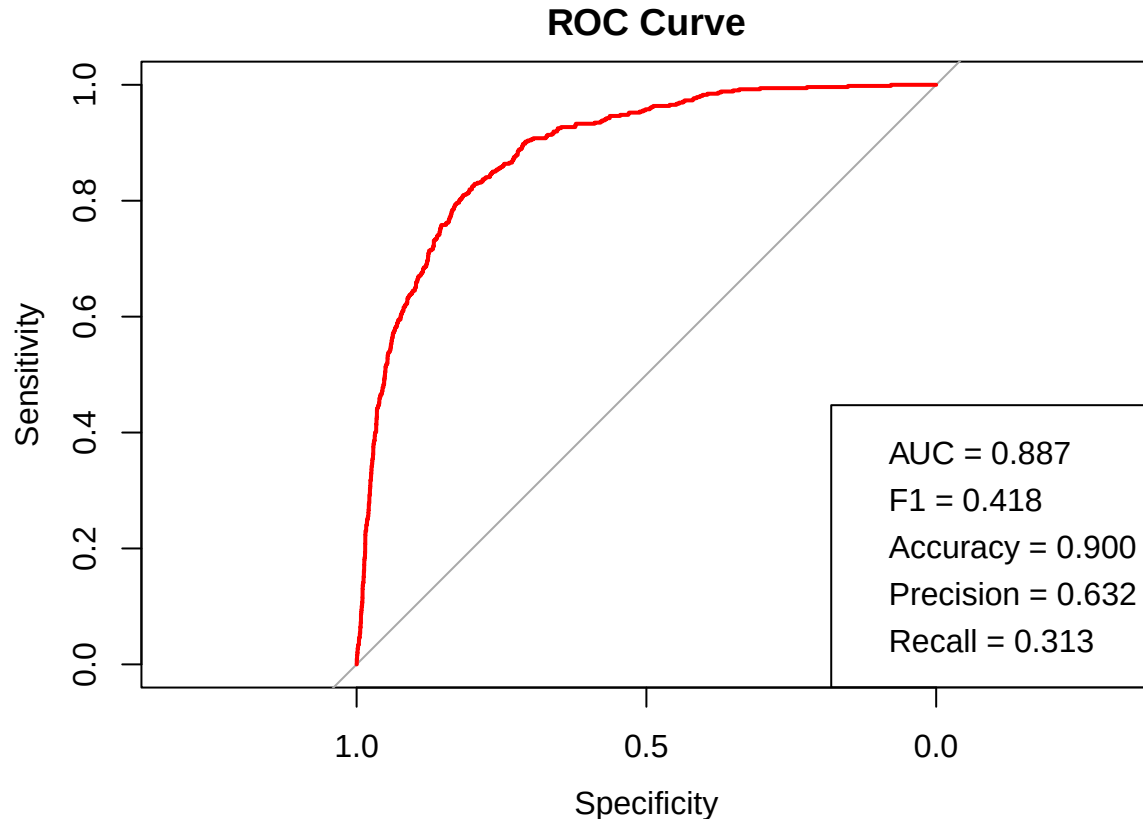
```
## [1] "jobblue-collar" "jobretired" "jobstudent" "maritalmarried"
## [5] "education.L" "housingyes" "loanyes" "contactunknown"
## [9] "month^4" "month^5" "month^6" "month^8"
## [13] "month^11" "duration" "poutcomeother" "pcomesuccess"
## [17] "poutcomeunknown"
```

Features that we have described as being weak were indeed zeroed down by lasso, while features like **housing** and **duration** remained. The **month** feature got expanded in 12 features (corresponding to 12 degrees of freedom) and 5 of those subfeatures have not only non-zero coef. but they have high absolute value.

Let us test selected features using CV.

```
lasso_data <- data[,c("y", "job", "marital", "education", "housing", "loan", "contact", "month", "duration")]
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = lasso_data)
```





We have been able to perform better with 90% accuracy and 63.2% precision. Thus we can say that our model overfitted the data.

Now we will try to the model trained without **month** feature.

```
build_formula <- function(arg_data, target = 'y', exclude = NULL) {
  predictors <- setdiff(names(arg_data), c(target, exclude))
  as.formula(paste(target, "~", paste(predictors, collapse = " + ")))
}

X <- model.matrix(build_formula(data, exclude = 'month'), data = data)[, -1] # remove intercept

y <- data$y
y <- ifelse(y == "yes", 1, 0)

model <- cv.glmnet(X, y, family = "binomial", alpha = 1)
coefs <- coef(model, s = 'lambda.1se') # or use s = "lambda.min" if using cv.glmnet
coefs <- as.matrix(coefs)

# Get variable names with non-zero coefficients
non_zero_vars <- rownames(coefs)[which(coefs != 0)]
non_zero_vars <- setdiff(non_zero_vars, "(Intercept)")

print(coefs)

##                lambda.1se
## (Intercept) -2.806758311
```



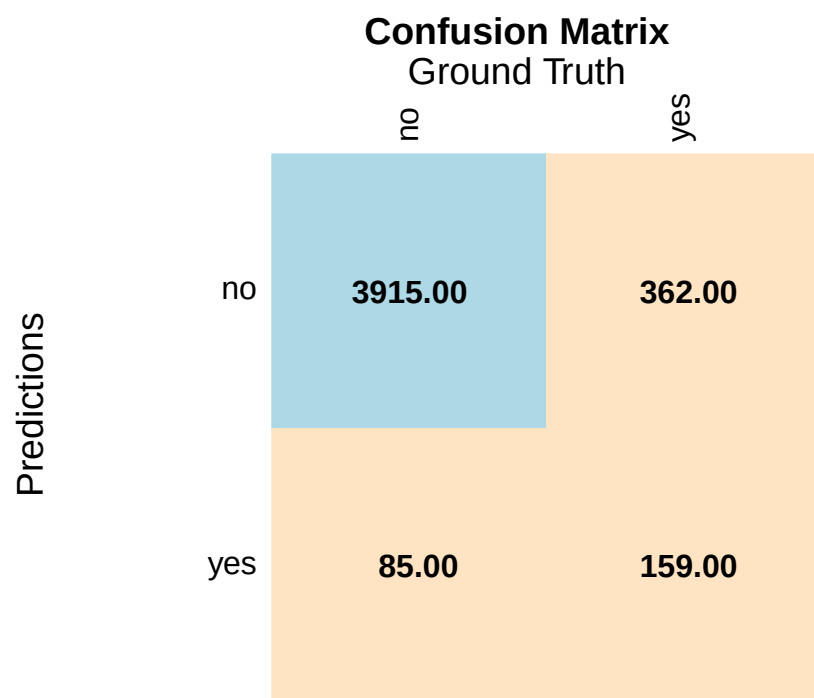
```
## age 0.000000000
## jobblue-collar -0.034282217
## jobentrepreneur 0.000000000
## jobhousemaid 0.000000000
## jobmanagement 0.000000000
## jobretired 0.227313388
## jobself-employed 0.000000000
## jobservices 0.000000000
## jobstudent 0.000000000
## jobtechnician 0.000000000
## jobunemployed 0.000000000
## jobunknown 0.000000000
## maritalmarried -0.003312757
## maritalsingle 0.000000000
## education.L 0.000000000
## education.Q 0.000000000
## education.C 0.000000000
## defaultyes 0.000000000
## balance 0.000000000
## housingyes -0.234323072
## loanyes -0.120390968
## contacttelephone 0.000000000
## contactunknown -0.581197216
## day 0.000000000
## duration 0.003165426
## campaign 0.000000000
## pdays 0.000000000
## previous 0.000000000
## poutcomeother 0.000000000
## poutcomesuccess 2.082604619
## poutcomeunknown -0.192419186
```

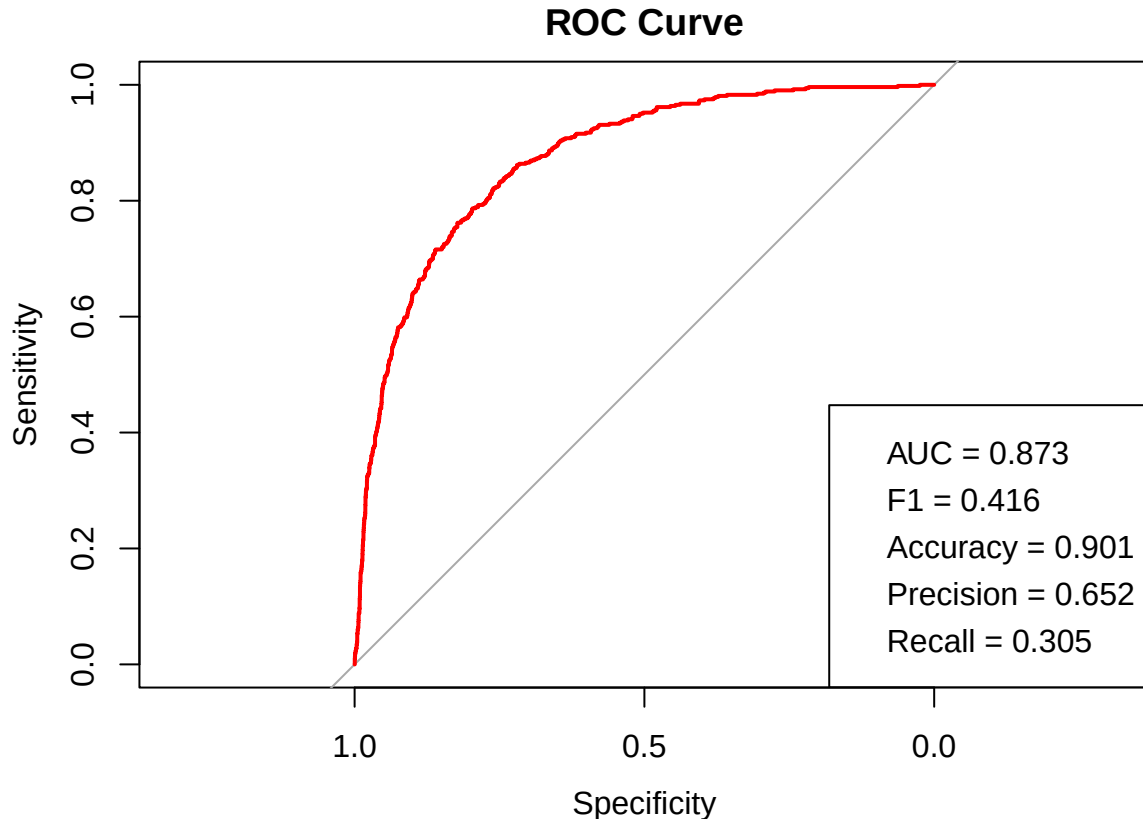
```
non_zero_vars
```

```
## [1] "jobblue-collar" "jobretired" "maritalmarried" "housingyes"
## [5] "loanyes" "contactunknown" "duration" "poutcomesuccess"
## [9] "poutcomeunknown"
```

An interesting observation is that without **month**, **education** and **day** features also disappeared.

```
lasso_data <- data[,c("y", "job", "marital", "housing", "loan", "contact", "duration", "poutcome")]
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = lasso_data)
```





We were able to bring accuracy and precision up once more. This time it was miniscule step, but nevertheless we obtained a better model.

### 1.3.2 Step with AIC heuristics

Next we will try to find the best subset using greedy search with AIC heuristics.

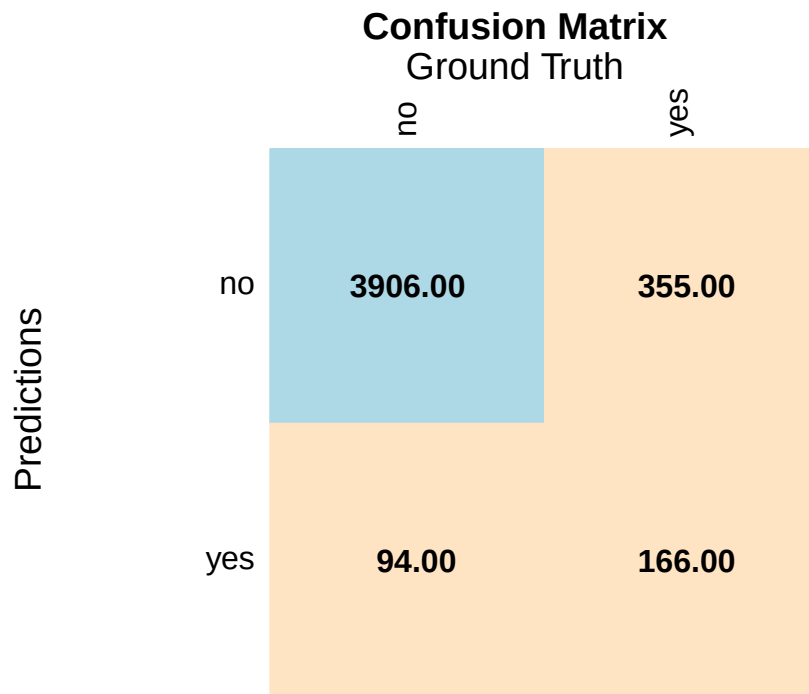
```
step(glm(y ~ ., data, family='binomial'), direction='both', trace=0)
```

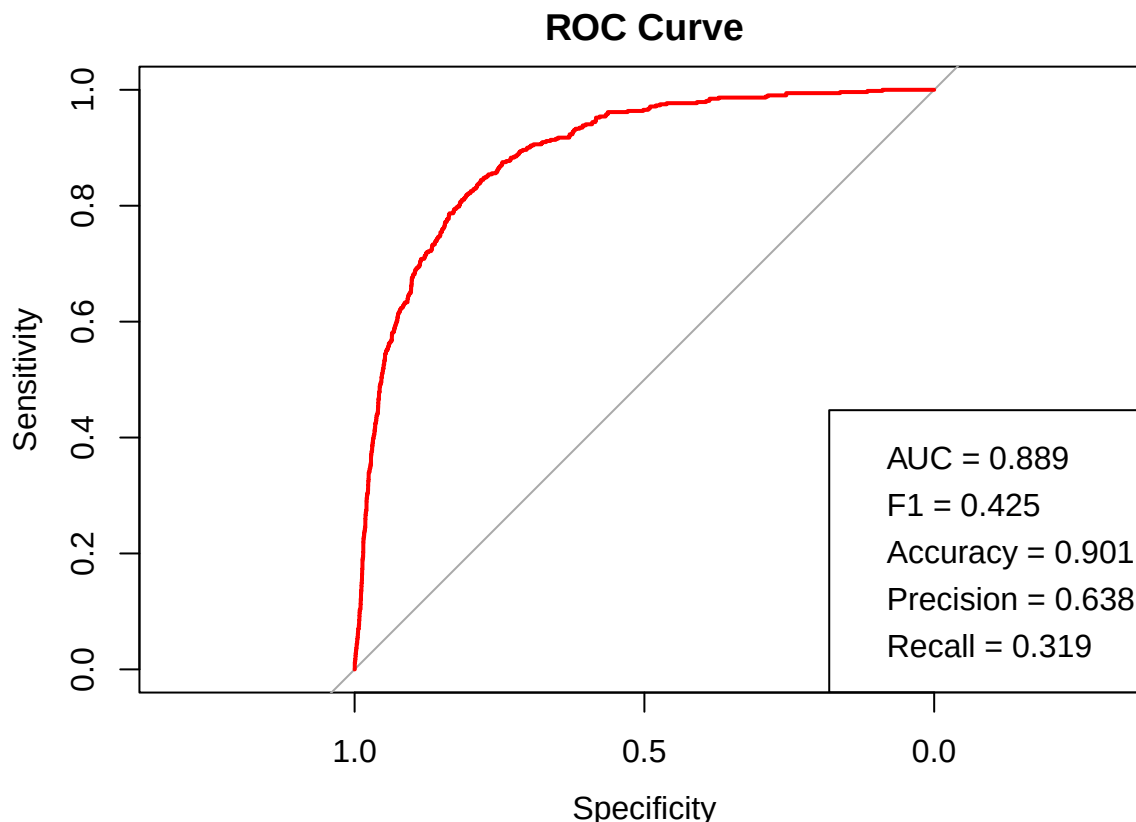
```
##
## Call:  glm(formula = y ~ marital + education + housing + loan + contact +
##       day + month + duration + campaign + poutcome, family = "binomial",
##       data = data)
##
## Coefficients:
##      (Intercept)    maritalmarried    maritalsingle    education.L
##      -2.573557      -0.510829      -0.318135      0.473929
##      education.Q    education.C    housingyes    loanyes
##      -0.014415      0.063460      -0.343417      -0.643090
## contacttelephone    contactunknown    day    month.L
##      -0.007710      -1.457777    0.016295    0.275732
##      month.Q    month.C    month^4    month^5
##      -0.432947    0.518895    -1.496841    0.848790
##      month^6    month^7    month^8    month^9
##      0.973761    0.746550    1.539579    -0.688630
##      month^10    month^11    duration    campaign
##      -0.048261    0.623386    0.004184    -0.072216
##      poutcomeother    poutcomesuccess    poutcomeunknown
```

```
## Degrees of Freedom: 4520 Total (i.e. Null); 4494 Residual
## Null Deviance: 3231
## Residual Deviance: 2196 AIC: 2250
```

Once again we see that **month**'s subfeatures get high coefficients. Step found a larger subset, than LASSO. Step is greedy, so it is harder to find extreme solutions such as LASSO.

```
step_data <- data[,c("y", "marital", "education", "housing", "loan", "contact", "day", "month", "duration")]
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = step_data)
```





We were able to find a good model. It has high precision and accuracy. Let us get rid of **month** to compare to engineered LASSO model.

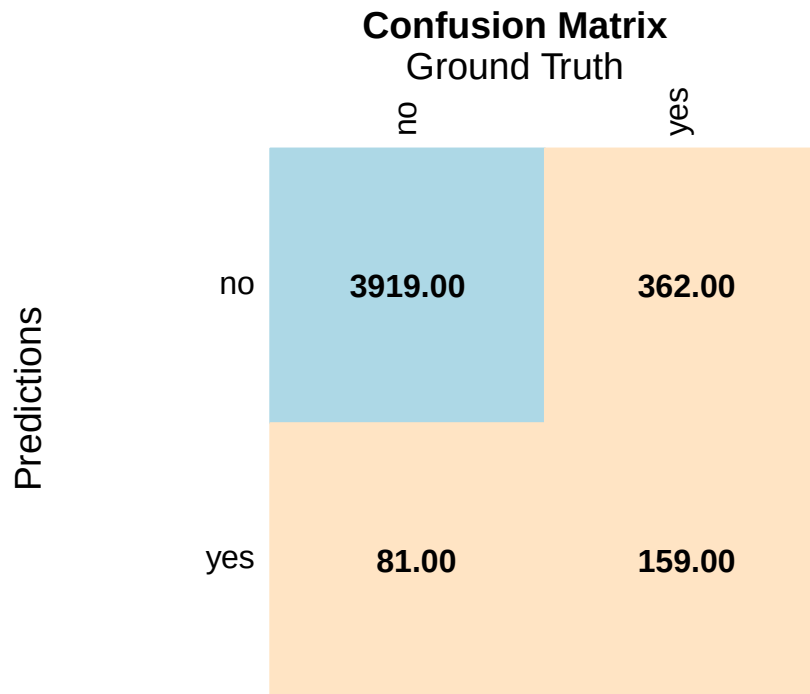
```
step(glm(y ~ ., data[,setdiff(names(data), c('month'))], family='binomial'), direction='both', trace=0)
```

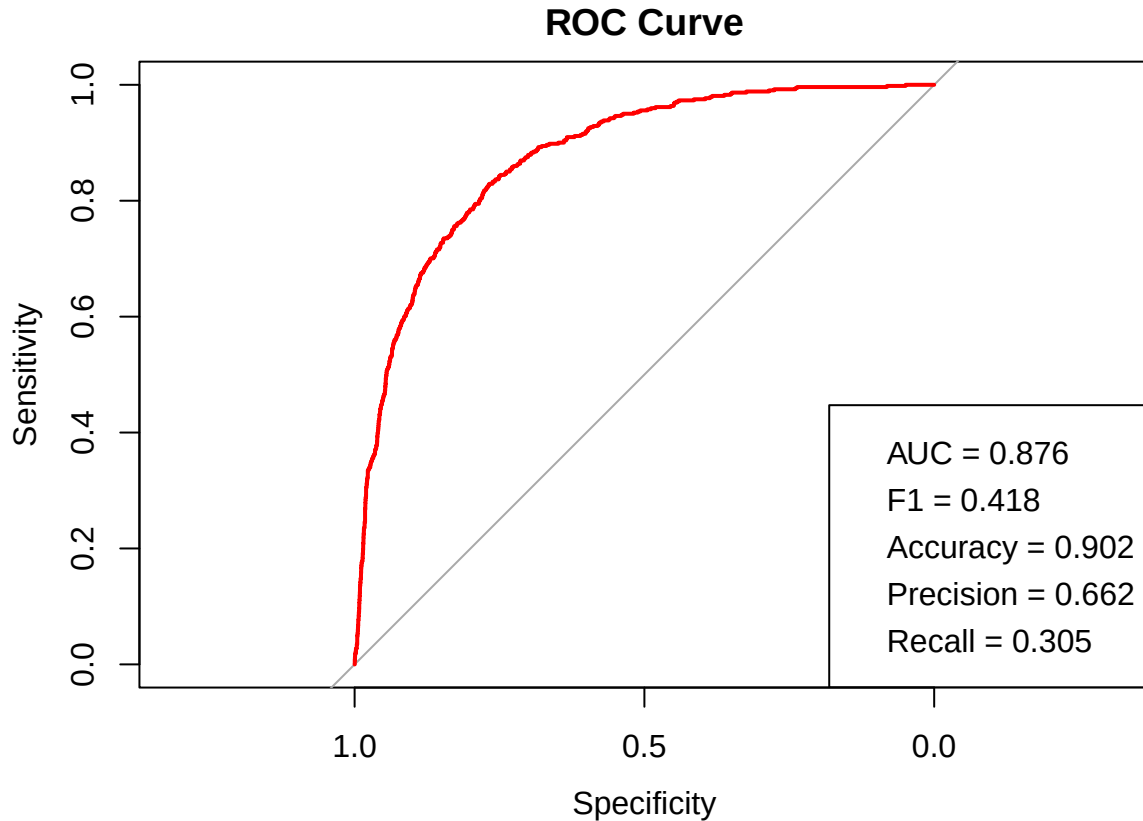
```
##
## Call: glm(formula = y ~ job + marital + education + housing + loan +
##   contact + duration + campaign + poutcome, family = "binomial",
##   data = data[, setdiff(names(data), c("month"))])
##
## Coefficients:
##   (Intercept)    jobblue-collar    jobentrepreneur    jobhousemaid
##      -2.377736      -0.449450      -0.430007      -0.303290
##   jobmanagement    jobretired    jobself-employed    jobservices
##      -0.103964       0.605486      -0.234316      -0.228378
##   jobstudent    jobtechnician    jobunemployed    jobunknown
##       0.513997      -0.250985      -0.694997       0.467686
##   maritalmarried    maritalsingle    education.L    education.Q
##      -0.439144      -0.209409       0.487303      -0.030979
##   education.C    housingyes    loanyes    contacttelephone
##       0.099334      -0.480192      -0.757944      -0.041167
##   contactunknown    duration    campaign    poutcomeother
##      -1.096033       0.004039      -0.074999       0.477239
##   poutcomesuccess    poutcomeunknown
##       2.410096      -0.227528
##
## Degrees of Freedom: 4520 Total (i.e. Null);  4495 Residual
```

```
## Null Deviance:      3231
## Residual Deviance: 2278  AIC: 2330
```

This time only **day** was zeroed out, while **education** remained.

```
step_data <- data[,c("y", "job", "marital", "education", "housing", "loan", "contact", "duration", "campus_location")]
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = step_data)
```





We obtain the best model yet with precision of 66%, though It has more features than LASSO selected.

### 1.3.3 Job Aggregation

All methods that we have used before showed that all **job** categories except *blue-collar* and *retired* have low or zero coefficients, so it will be meaningful to test whether we can use smaller model without loss of precision.

```
data.specific_job <- data
data.specific_job[!(data$job == 'blue-collar' | data$job == 'retired'),"job"] <- "unknown"
head(data.specific_job)
```

```
##   age      job marital education default balance housing loan  contact day
## 1  30   unknown married  primary      no   1787      no   no cellular  19
## 2  33   unknown married secondary     no   4789     yes  yes cellular  11
## 3  35   unknown single  tertiary     no   1350     yes   no cellular  16
## 4  30   unknown married  tertiary     no   1476     yes  yes unknown   3
## 5  59 blue-collar married secondary     no     0     yes   no unknown   5
## 6  35   unknown single  tertiary     no    747      no   no cellular  23
##  month duration campaign pdays previous poutcome y
## 1  oct         79         1     -1         0 unknown no
## 2  may        220         1    339         4 failure no
## 3  apr        185         1    330         1 failure no
## 4  jun        199         4     -1         0 unknown no
## 5  may        226         1     -1         0 unknown no
## 6  feb        141         2    176         3 failure no
```

Firstly we will run LASSO with joint jobs categories.

```

X <- model.matrix(build_formula(data.specific_job, exclude = 'month'), data = data)[, -1] # remove int
y <- data$y
y <- ifelse(y == "yes", 1, 0)

model <- cv.glmnet(X, y, family = "binomial", alpha = 1)
coefs <- coef(model, s = 'lambda.1se') # or use s = "lambda.min" if using cv.glmnet
coefs <- as.matrix(coefs)

# Get variable names with non-zero coefficients
non_zero_vars <- rownames(coefs)[which(coefs != 0)]
non_zero_vars <- setdiff(non_zero_vars, "(Intercept)")

print(coefs)

```

```

##                lambda.1se
## (Intercept)    -2.806758311
## age            0.000000000
## jobblue-collar -0.034282217
## jobentrepreneur 0.000000000
## jobhousemaid    0.000000000
## jobmanagement   0.000000000
## jobretired      0.227313388
## jobself-employed 0.000000000
## jobservices     0.000000000
## jobstudent      0.000000000
## jobtechnician   0.000000000
## jobunemployed   0.000000000
## jobunknown      0.000000000
## maritalmarried  -0.003312757
## maritalsingle   0.000000000
## education.L     0.000000000
## education.Q     0.000000000
## education.C     0.000000000
## defaultyes      0.000000000
## balance         0.000000000
## housingyes      -0.234323072
## loanyes         -0.120390968
## contacttelephone 0.000000000
## contactunknown  -0.581197216
## day            0.000000000
## duration        0.003165426
## campaign        0.000000000
## pdays           0.000000000
## previous        0.000000000
## poutcomeother   0.000000000
## poutcomesuccess 2.082604619
## poutcomeunknown -0.192419186

```

```
non_zero_vars
```

```

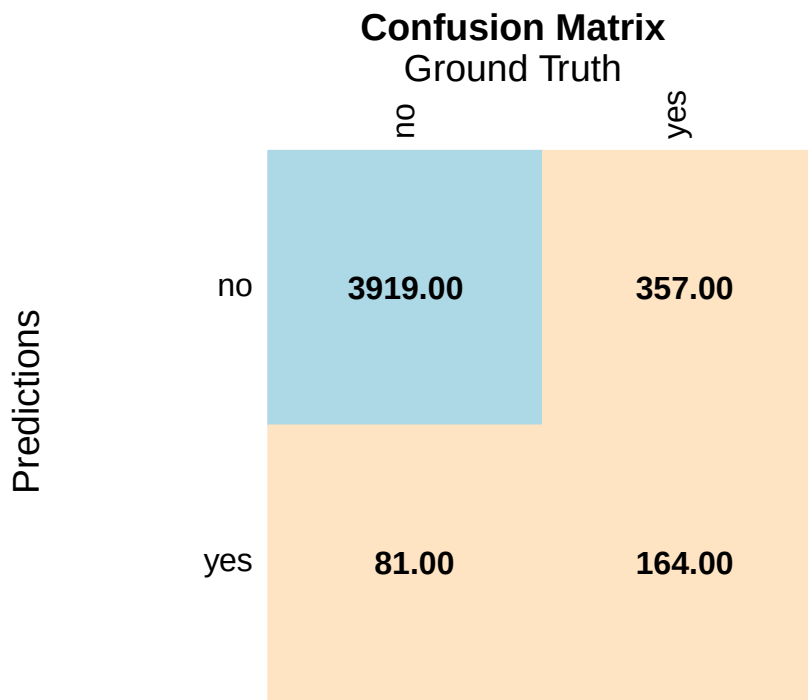
## [1] "jobblue-collar" "jobretired"      "maritalmarried"  "housingyes"
## [5] "loanyes"        "contactunknown"  "duration"        "poutcomesuccess"
## [9] "poutcomeunknown"

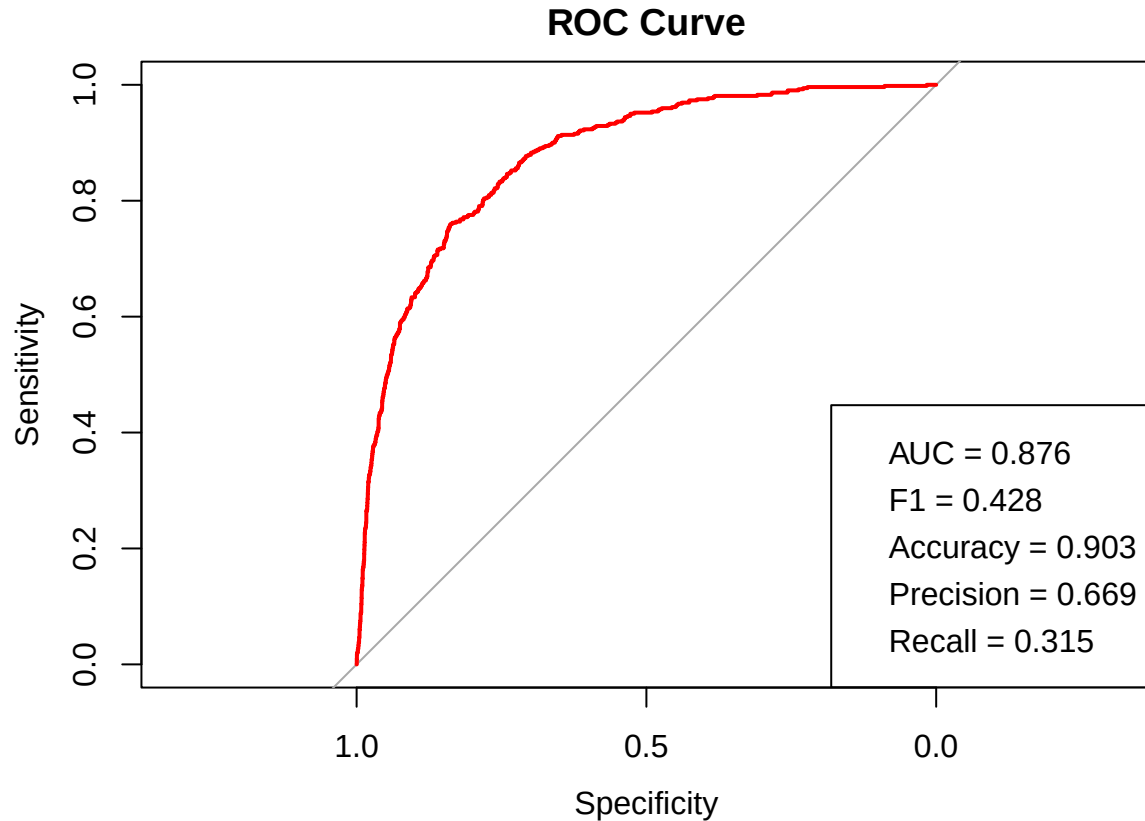
```



The feature selection resulted in the very same features as before, let us run CV.

```
lasso_data <- data.specific_job[,c("y", "job", "marital", "housing", "loan", "contact", "duration", "po  
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = lasso_data)
```





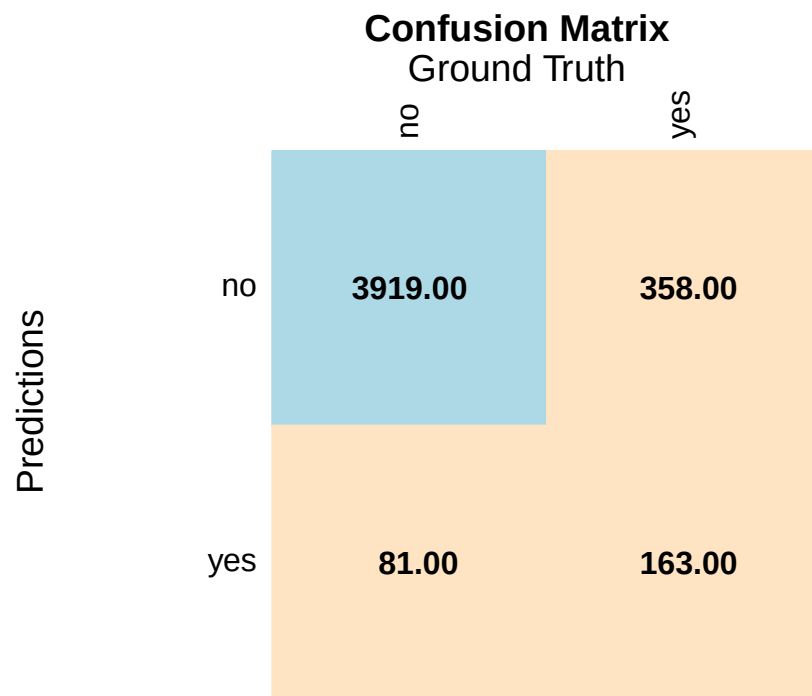
We have also obtained better results regarding precision. Let us run the same procedure with step function.

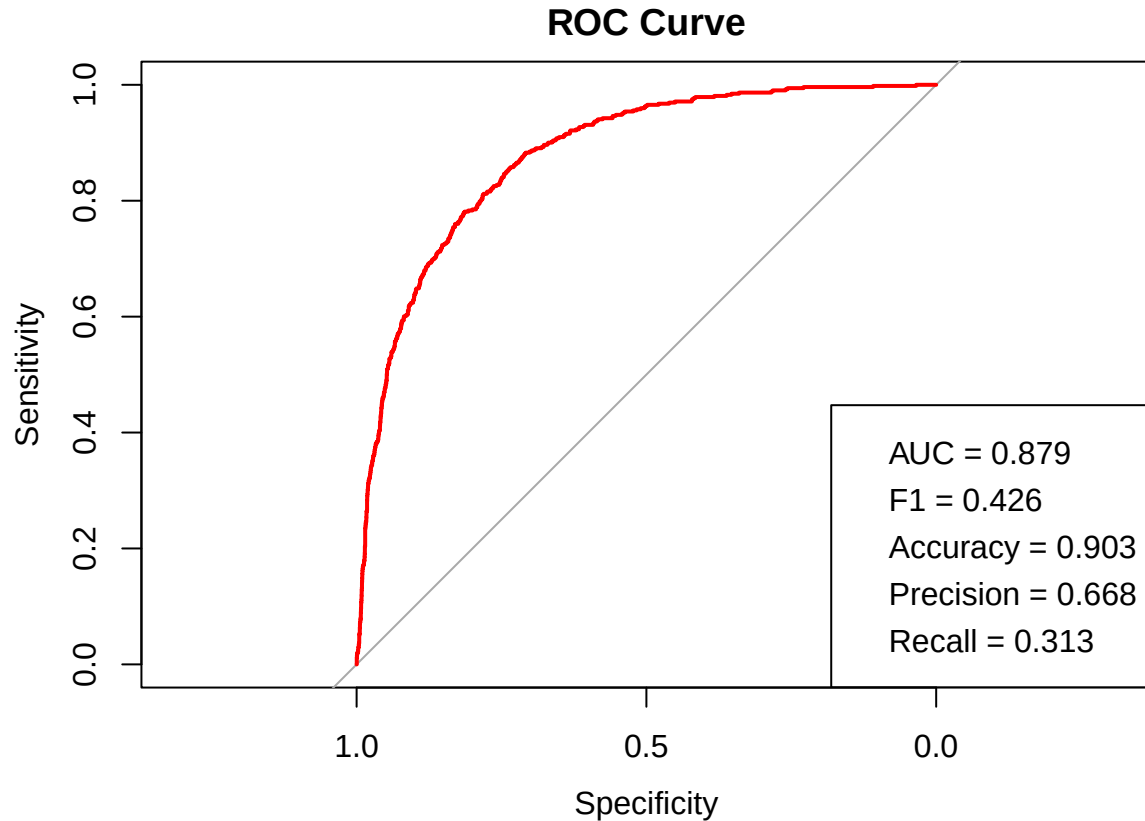
```
step(glm(y ~ ., data=data.specific_job[,setdiff(names(data), c('month'))] , family = binomial), direction = "backward")
```

```
##
## Call:  glm(formula = y ~ job + marital + education + housing + loan +
##       contact + duration + campaign + poutcome, family = binomial,
##       data = data.specific_job[, setdiff(names(data), c("month"))])
##
## Coefficients:
##      (Intercept)      jobretired      jobunknown      maritalmarried
##      -2.789462         1.052547         0.274764        -0.431151
##      maritalsingle      education.L      education.Q      education.C
##      -0.149293         0.428147         0.038811         0.057484
##      housingyes         loanyes      contacttelephone      contactunknown
##      -0.493092        -0.763278        -0.042073        -1.104398
##      duration          campaign      poutcomeother      poutcomesuccess
##      0.004008        -0.074595         0.495816         2.416749
##      poutcomeunknown
##      -0.237666
##
## Degrees of Freedom: 4520 Total (i.e. Null);  4504 Residual
## Null Deviance:      3231
## Residual Deviance: 2289  AIC: 2323
```

The situation remains the same, the features chosen by step has not changed.

```
step_data <- data.specific_job[,c("y", "job", "marital", "education", "housing", "loan", "contact", "du
cv_evaluate_model(glm, glm_predict_func, family=binomial, arg_data = step_data)
```





This model also beats its previous version, though precision is slightly lower.

## 1.4 Final model

After all feature selection and engineering we choose LASSO model with joint **job** and excluded **month** feature.

```
final_model <- glm(y ~ job + marital + housing + loan + contact + poutcome + duration, data=data.specific_job)
summary(final_model)
```

```
##
## Call:
## glm(formula = y ~ job + marital + housing + loan + contact +
##      poutcome + duration, family = binomial, data = data.specific_job)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -2.9046849   0.2794213 -10.395 < 2e-16 ***
## jobretired      1.1080491   0.2479450   4.469 7.86e-06 ***
## jobunknown      0.4148859   0.1630157   2.545 0.01093 *
## maritalmarried  -0.4384189   0.1666483  -2.631 0.00852 **
## maritalsingle  -0.1201637   0.1808508  -0.664 0.50641
## housingyes     -0.4952104   0.1190575  -4.159 3.19e-05 ***
## loanyes        -0.7790165   0.1914404  -4.069 4.72e-05 ***
## contacttelephone -0.1048519   0.2169909  -0.483 0.62895
## contactunknown -1.1135127   0.1741227  -6.395 1.61e-10 ***
## poutcomeother   0.4736040   0.2553980   1.854 0.06369 .
## pcomesuccess    2.3926894   0.2530984   9.454 < 2e-16 ***
```

```
## poutcomeunknown -0.3034844 0.1707815 -1.777 0.07556 .
## duration 0.0039657 0.0001917 20.683 < 2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 3231.0 on 4520 degrees of freedom
## Residual deviance: 2305.4 on 4508 degrees of freedom
## AIC: 2331.4
##
## Number of Fisher Scoring iterations: 6
```

We see that *single* marital status, *telephone* contact and *other, unknown* previous outcome are unstable, their effect on the model is not too large, due to their small coefficients, so we will not confront this.

## 1.5 Assumption Verification

Logistic regression assumes (besides i.i.d. and categorical target) linearity of continuous logits, no outliers and no multicollinearity. We will check those assumptions with Box-Tidwell test, Cook's distance and GVIF metric.

### 1.5.1 Linearity of Continuous Variable Logits

We will run Box-Tidwell test that determines what power transformation makes logit linear, we test that power transformation has exponent of 1. The only continuous feature that we have is **duration**.

```
data.specific_job.numeric <- data.specific_job
data.specific_job.numeric$y <- ifelse(data.specific_job$y == "yes", 1, 0)
boxTidwell(y ~ duration, data = data.specific_job.numeric)
```

```
## MLE of lambda Score Statistic (t) Pr(>|t|)
## 0.79334 -4.4352 9.419e-06 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## iterations = 5
```

We must reject the null hypothesis, we will apply the power transformation with MLE estimated exponent to **duration**.

```
data.specific_job.numeric$duration_transformed <- data.specific_job$duration ^ lambda
car::boxTidwell(y ~ duration_transformed, data = data.specific_job.numeric)
```

```
## MLE of lambda Score Statistic (t) Pr(>|t|)
## 0.99984 -0.0022 0.9982
##
## iterations = 0
```

We have passed the test, let us examine fixed model.

```
final_model <- glm(y ~ job + marital + housing + loan + contact + poutcome + I(duration ^ lambda), data = data.specific_job, family = binomial)
summary(final_model)
```

```
##
## Call:
## glm(formula = y ~ job + marital + housing + loan + contact +
## poutcome + I(duration^lambda), family = binomial, data = data.specific_job)
```

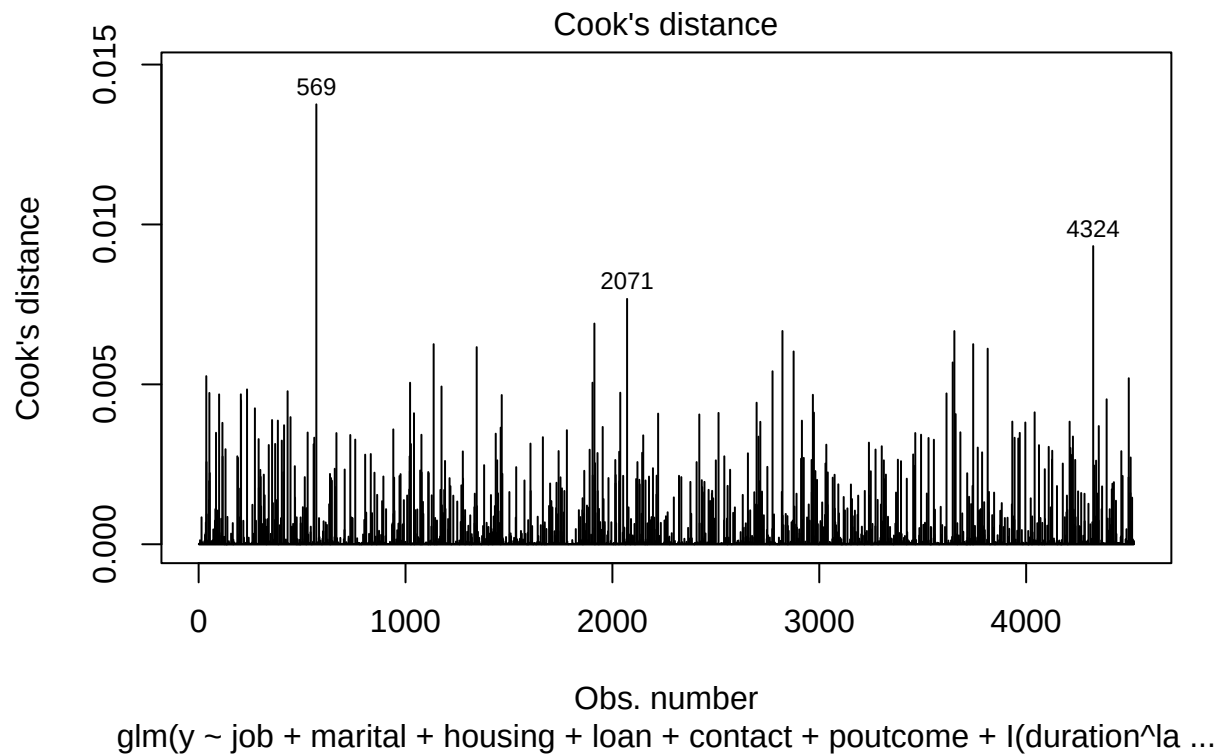
```
##
## Coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -3.3835606  0.2872977 -11.777 < 2e-16 ***
## jobretired     1.1048991  0.2495903   4.427 9.56e-06 ***
## jobunknown     0.4246159  0.1629615   2.606 0.00917 **
## maritalmarried -0.4343403  0.1678107  -2.588 0.00965 **
## maritalsingle  -0.1134125  0.1821536  -0.623 0.53353
## housingyes     -0.4986848  0.1196789  -4.167 3.09e-05 ***
## loanyes        -0.7780515  0.1913010  -4.067 4.76e-05 ***
## contacttelephone -0.0828720  0.2191188  -0.378 0.70528
## contactunknown -1.1037494  0.1734015  -6.365 1.95e-10 ***
## poutcomeother   0.4755413  0.2577277   1.845 0.06502 .
## poutcomesuccess 2.3874747  0.2555046   9.344 < 2e-16 ***
## poutcomeunknown -0.3073699  0.1719614  -1.787 0.07387 .
## I(duration^lambda) 0.0185148  0.0008628  21.459 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 3231.0  on 4520  degrees of freedom
## Residual deviance: 2270.4  on 4508  degrees of freedom
## AIC: 2296.4
##
## Number of Fisher Scoring iterations: 6
```

The coefficients did not change, but AIC decreased as well as deviances.

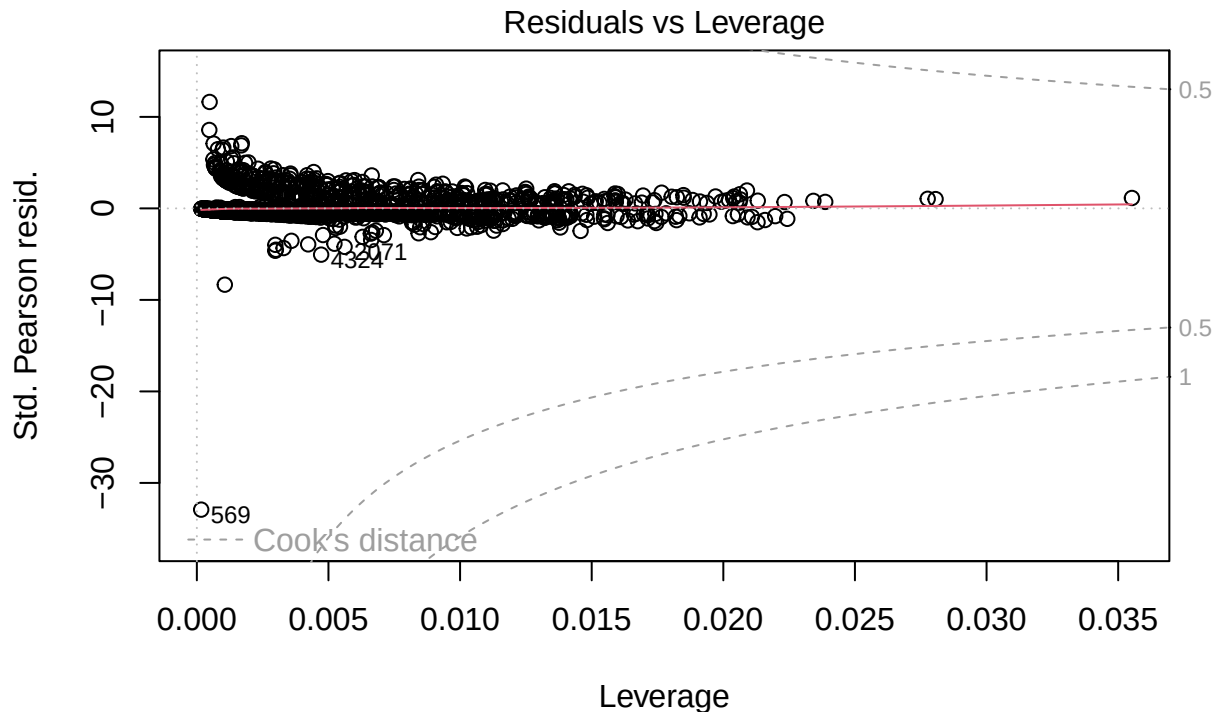
### 1.5.2 Outliers

We will check for outliers in data set using Cook's d.

```
plot(final_model, which=4)
```



```
plot(final_model, which=5)
```



`glm(y ~ job + marital + housing + loan + contact + poutcome + I(duration^la ...`

None of the datapoints lie outside of Cook's d range, so formally we have no outliers. On the other hand sample 569 has the largest distance, though it has low leverage we shall take a look at it.

```
data[569,]
```

```
##      age      job marital education default balance housing loan  contact day
## 569  59 unemployed married  primary      no        0      no  no cellular  30
##      month duration campaign pdays previous poutcome  y
## 569   jan     3025         2    -1         0 unknown no
```

The biggest quirk of this sample is long call **duration** (3000s=50 minutes), yet it still has not subscribed to the deposit. We will remove this sample, because we have enough data and its low leverage guarantee that it will not hurt our model.

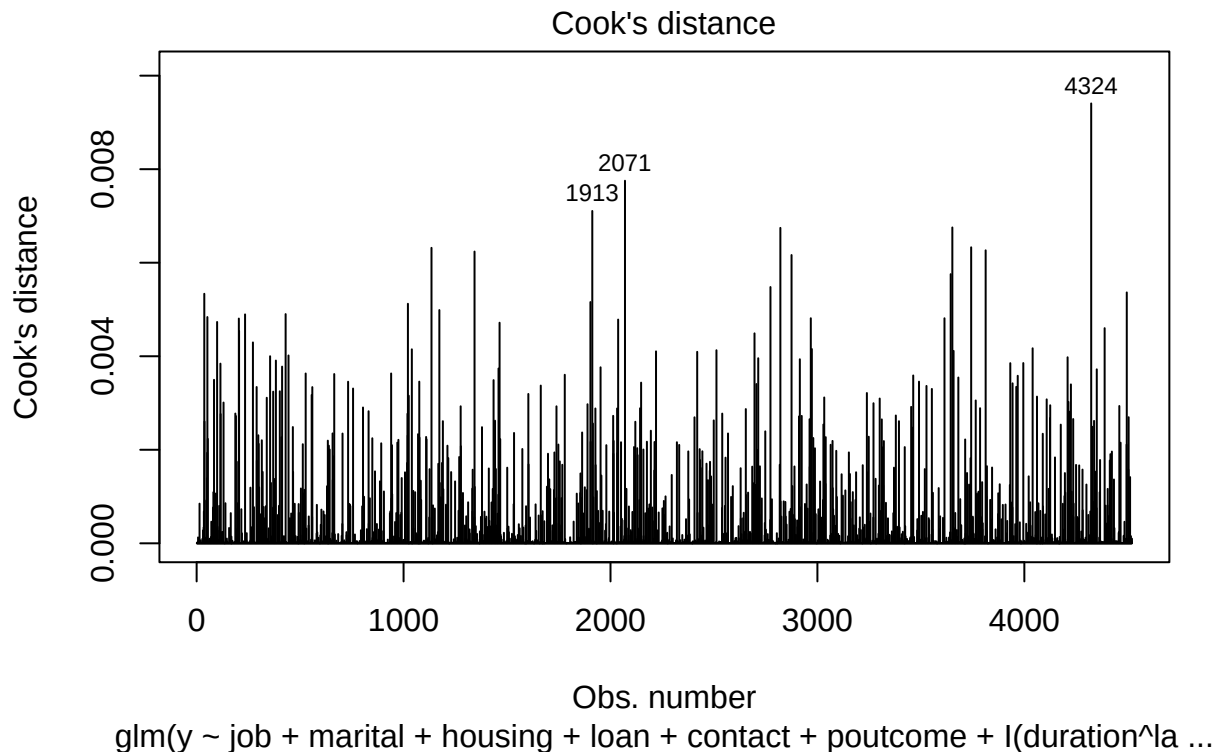
```
final_model <- glm(y ~ job + marital + housing + loan + contact + poutcome + I(duration ^ lambda), data = data)
summary(final_model)
```

```
##
## Call:
## glm(formula = y ~ job + marital + housing + loan + contact +
##      poutcome + I(duration^lambda), family = binomial, data = data.specific_job[-569,
##      ])
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -3.4234144   0.2887112  -11.858  < 2e-16 ***
## jobretired      1.1079931   0.2505248   4.423 9.75e-06 ***
## jobunknown      0.4337103   0.1636434   2.650 0.00804 **
## maritalmarried  -0.4286682   0.1684699  -2.544 0.01094 *
```

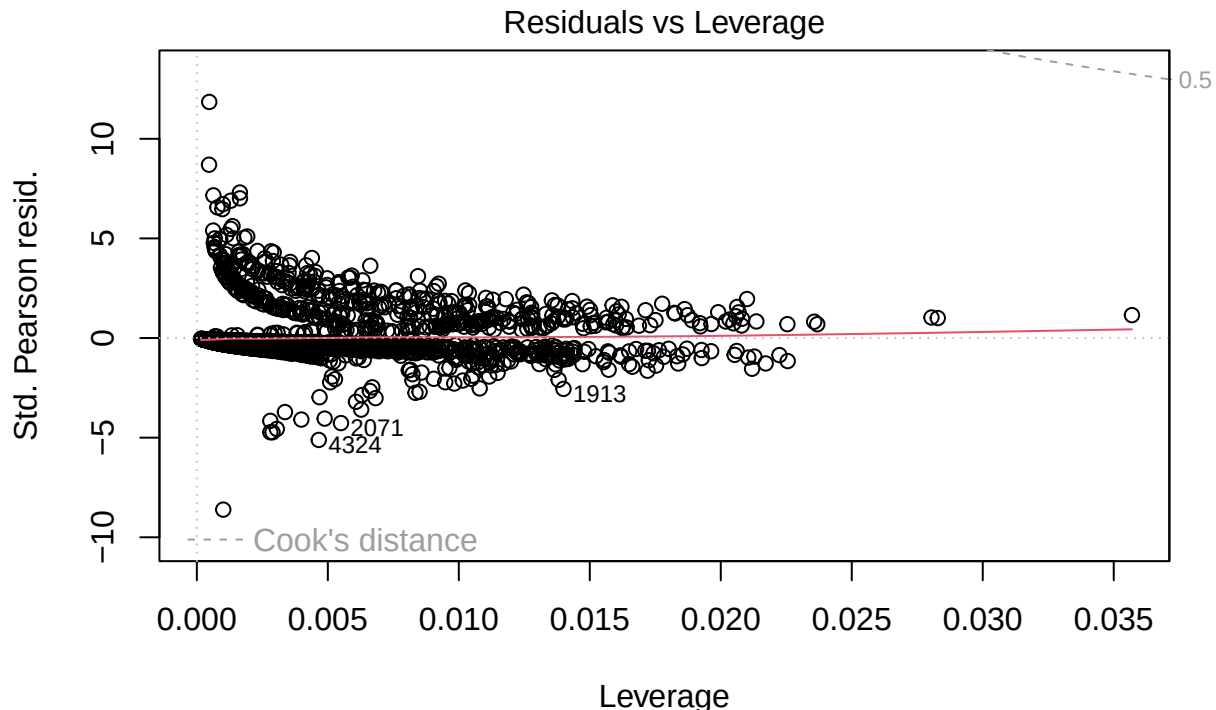


```
## maritalsingle      -0.1144606  0.1829358  -0.626  0.53152
## housingyes         -0.5092094  0.1200770  -4.241  2.23e-05 ***
## loanyes            -0.7889140  0.1921775  -4.105  4.04e-05 ***
## contacttelephone   -0.0867686  0.2201306  -0.394  0.69346
## contactunknown     -1.1175292  0.1742463  -6.414  1.42e-10 ***
## poutcomeother       0.4754379  0.2587739   1.837  0.06617 .
## pcomesuccess        2.3901548  0.2561809   9.330  < 2e-16 ***
## poutcomeunknown    -0.3070400  0.1725872  -1.779  0.07523 .
## I(duration^lambda)  0.0188705  0.0008714  21.656  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 3230.8  on 4519  degrees of freedom
## Residual deviance: 2256.2  on 4507  degrees of freedom
## AIC: 2282.2
##
## Number of Fisher Scoring iterations: 6
```

```
plot(final_model, which=4)
```



```
plot(final_model, which=5)
```



`glm(y ~ job + marital + housing + loan + contact + poutcome + I(duration^la ...`

With no strong outliers we can proceed to the last assumption.

### 1.5.3 Multicollinearity

Logistic regression is a linear model and can suffer a lot from multicollinearity. Base R implementation does not use any regularization, so it can destroy our model. Luckily LASSO usually removes collinear features, so we should be good to go.

```
vif(final_model)
```

```
##              GVIF Df GVIF^(1/(2*Df))
## job          1.162905 2          1.038451
## marital      1.059598 2          1.014578
## housing      1.147895 1          1.071399
## loan         1.018021 1          1.008970
## contact      1.169395 2          1.039897
## poutcome     1.131937 3          1.020870
## I(duration^lambda) 1.087601 1          1.042881
```

All adjusted GVIF values are way lower than 2, so we can assume that no multicollinearity is present in the data. Let us plot last ROC curve and confusion matrix.

```
y_true_all <- data.specific_job$y
y_prob_all <- glm_predict_func(final_model, data.specific_job)
y_pred_all <- ifelse(y_prob_all > 0.5, "yes", "no")
y_pred_all <- factor(y_pred_all, levels = c("no", "yes"))

cm <- table(Predicted = y_pred_all, Actual = y_true_all)
```

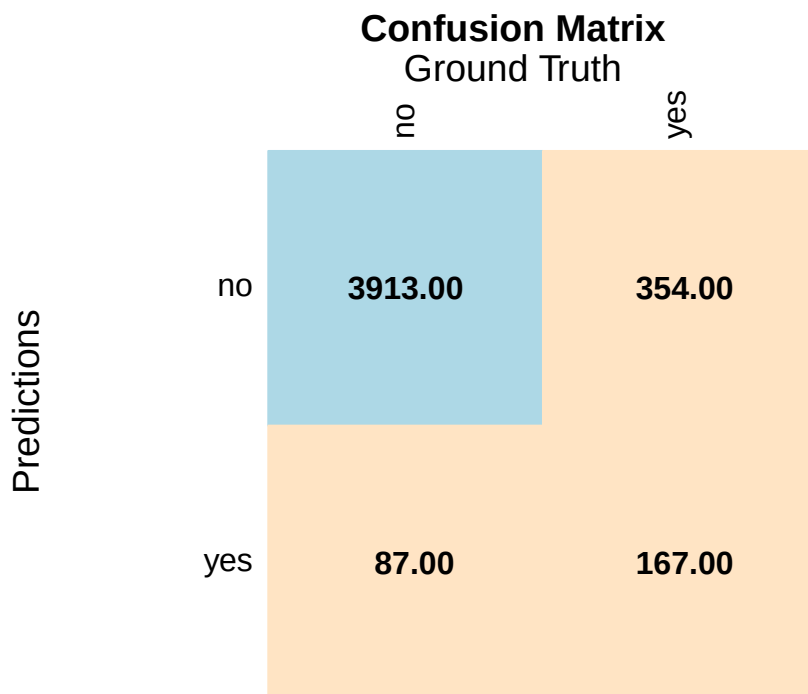
```

tp <- cm["yes", "yes"]
tn <- cm["no", "no"]
fp <- cm["yes", "no"]
fn <- cm["no", "yes"]

accuracy <- (tp + tn) / sum(cm)
precision <- tp / (tp + fp)
recall <- tp / (tp + fn)
f1 <- 2 * precision * recall / (precision + recall)

corrplot(
  cm,
  is.corr = F, method = "color", tl.col = "black", addCoef.col = "black",
  xlab = "Ground Truth", ylab = "Prediction", mar=c(3,0,3,0),
  title = "Confusion Matrix", cl.pos = 'n', col=c('bisque', 'lightblue')
)
mtext("Ground Truth", side = 3, line=1, cex = 1.2)
mtext("Predictions", side = 2, line=0, cex = 1.2)

```



```

roc_obj <- roc(y_true_all, y_prob_all, levels=c("no", "yes"), direction='<')
plot(roc_obj, col = "red", main = "ROC Curve")

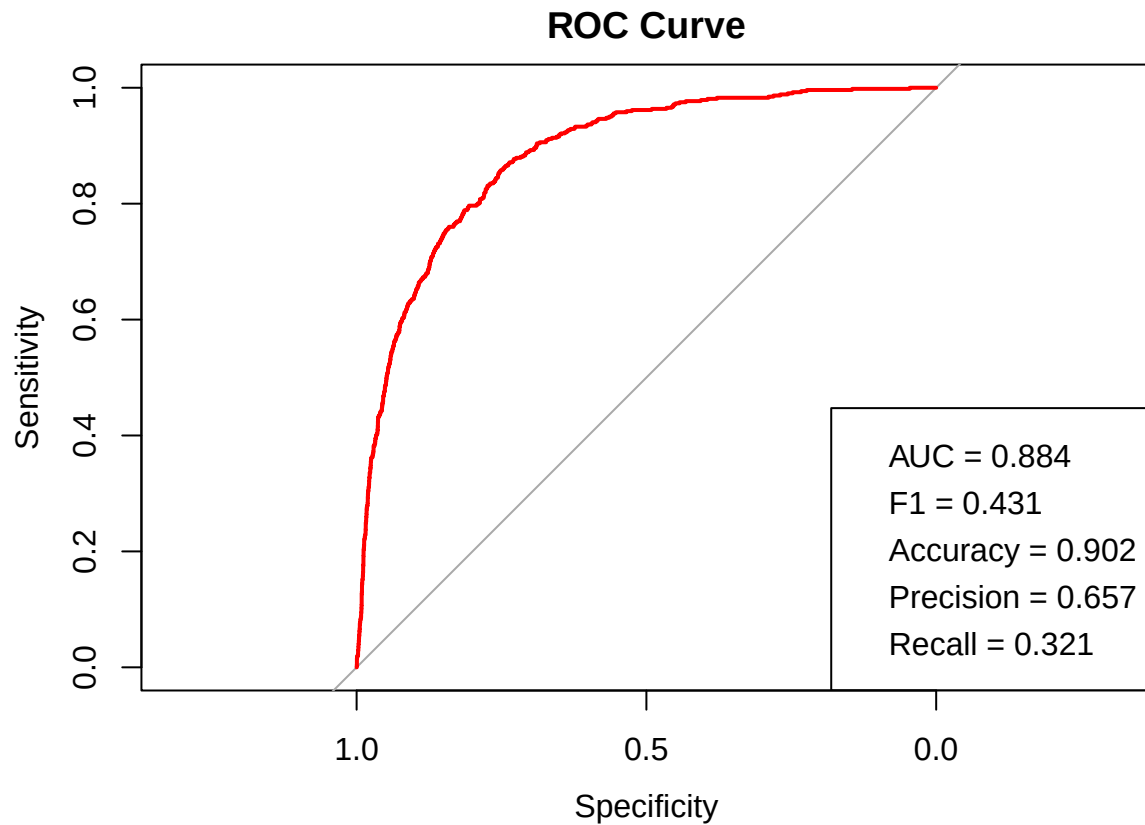
legend(
  "bottomright",

```

```

legend = c(
  sprintf("AUC = %.3f", auc(roc_obj)),
  sprintf("F1 = %.3f", f1),
  sprintf("Accuracy = %.3f", accuracy),
  sprintf("Precision = %.3f", precision),
  sprintf("Recall = %.3f", recall)
)
)

```



These are purely training values, though we still have large enough precision and good accuracy. The model still favors negative class, though not as sharply as before.

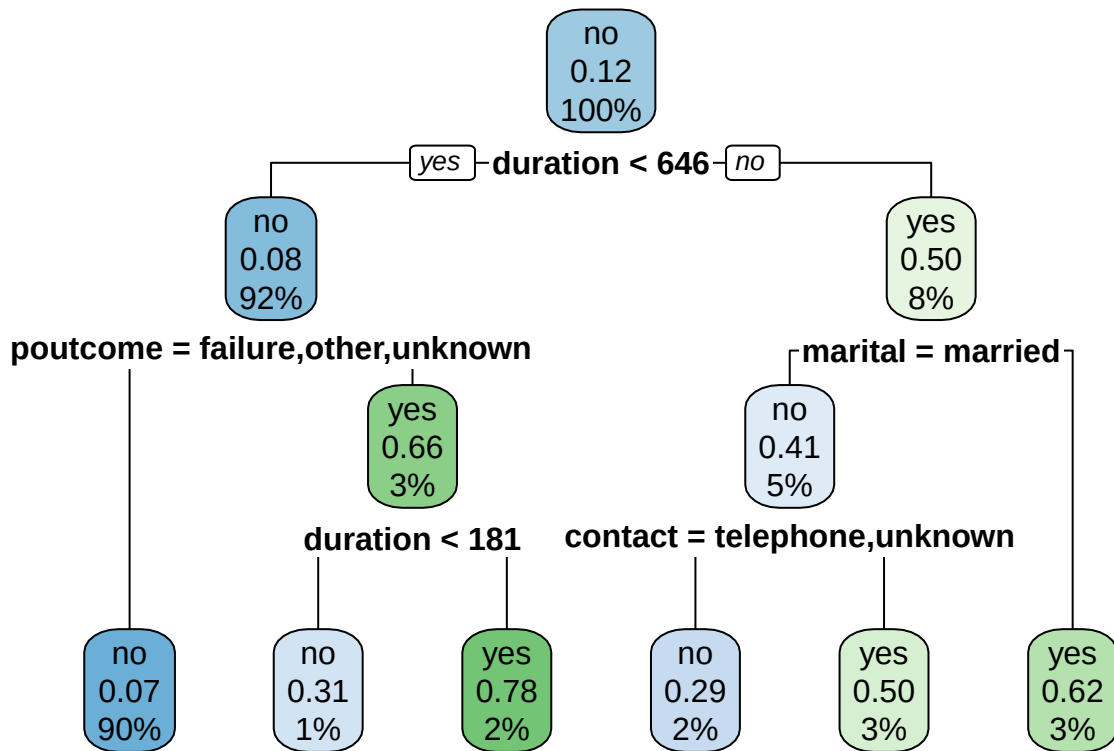
## 1.6 Final Decision Tree

Lastly we will train decision tree for further rule extraction.

```

data.lasso_specific_job <- data.specific_job[,c("y", "job", "marital", "housing", "loan", "contact", "d
tree_model <- rpart(y ~ ., data.lasso_specific_job, maxdepth=3, cp=0)
rpart.plot(tree_model)

```



Most of the leaf nodes that predict positive class have close to 50% positive population, so the rules extracted from this tree should be taken with a grain of salt.

## 1.7 Wrap up

Now we will save our models.

```
saveRDS(final_model, file=here::here("models", "log_regression.rds"))
saveRDS(tree_model, file=here::here("models", "decision_tree.rds"))
```

In this notebook we have done feature selection both manual and automatic. Next step is to use all the collected knowledge to design target profile.